

What are methods?

INTERMEDIATE JAVA



Jim White
Java Developer

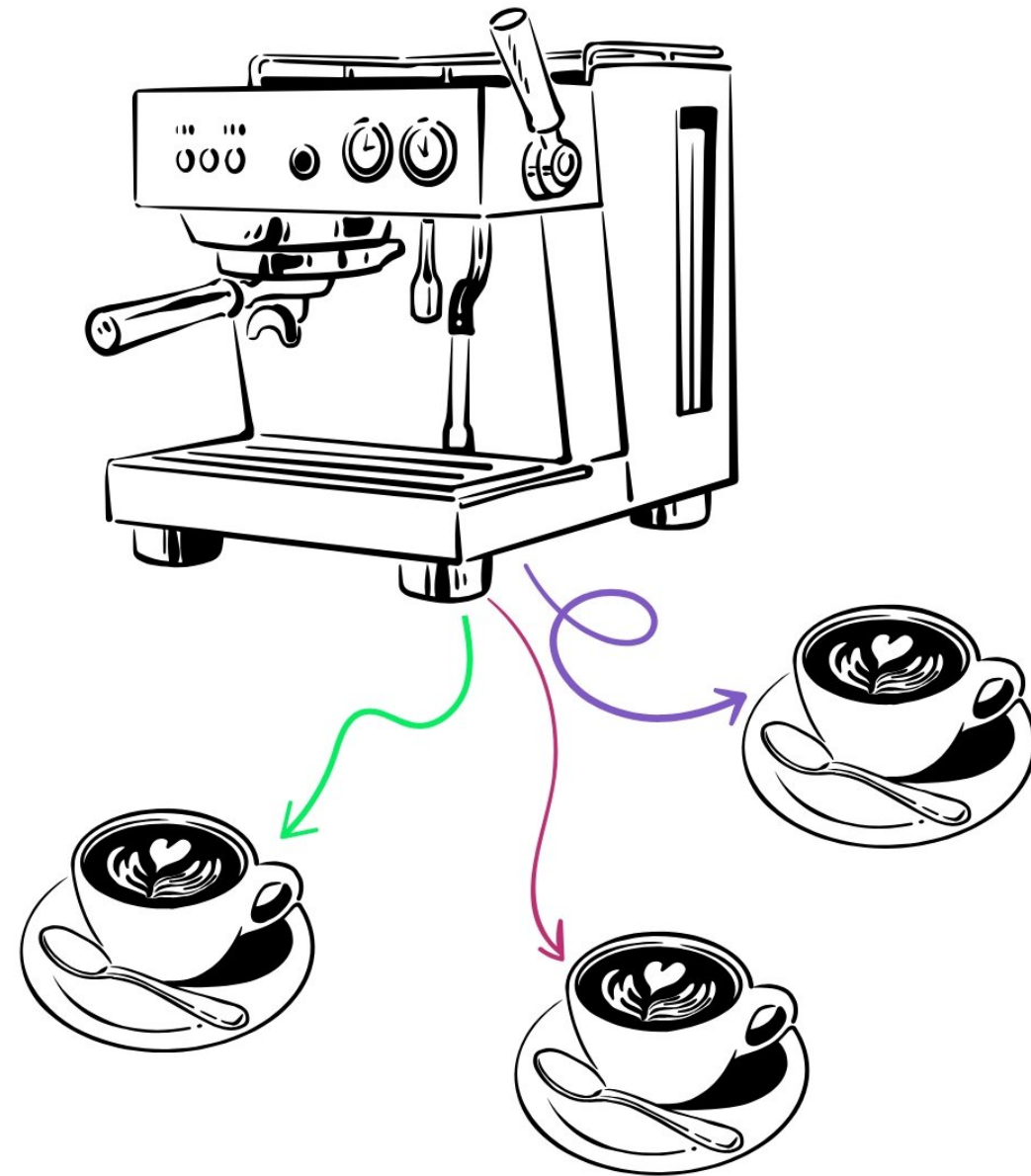
What you'll learn

- Methods as reusable blocks of code
 - Custom methods
 - Parameters
- Control flow
 - `if-else` statements
 - `switch` statements
 - Loops
 - `for` loops
 - `while` loops
- Workflow



What is a method?

- Block of code performing a specific task
- Allows reusing of code



Methods we've already seen

- Built-in types have their own methods

For example, `String` :

- `.length()`
- `.toLowerCase()` , ...
- and more methods to come!

.contains()

- Check if `String` contains a specific substring
- Returns `true` or `false`

```
"Java".contains("av"); // Returns true
```

.charAt()

- Returns character at specified place

```
"Java".charAt(1); // Returns 'a'
```

Remember, Java starts counting at 0!

.substring()

- Returns part of `String` , starting at `start` and stopping before `end` :

```
String.substring(start, end)
```

```
"Java".substring(1,3); // Return "av"
```


.equals()

- Compares values of `String` s
- Returns `true` if values are the same

```
String word = "Java";  
word.equals("Java"); // Returns true
```

Reminder: `==` can give unexpected results with `String` s

Summary

```
word = "Java";

// .contains()
"Java".contains("v"); // Returns true
word.contains("w"); // Returns false

// .charAt()
"Java".charAt(0); // Returns 'J'
word.charAt(1); // Returns 'a'
```

```
// .substring()
"Java".substring(1,3); // Returns "av"
word.substring(0,1); // Returns "J"

// .equals()
"Java".equals(word); // Returns true

// The following may
// or may not return true
boolean output = "Java" == word;
```

Let's practice!

INTERMEDIATE JAVA

Creating your own methods

INTERMEDIATE JAVA



Jim White
Java Developer

Why we use methods

- Great for repeated tasks
- Reusing of code
- Avoid repetition (and potential mistakes)
- Easier to update and fix mistakes
- **Don't Repeat Yourself (DRY)**

How to define methods

```
returnType methodName() {  
  
    // Code to be run  
  
}
```

void method

- Only performs an *action*, doesn't return anything
- `void` means *nothing*

```
void sayHello(){  
    System.out.println("Hello there!");  
}
```

Method that produces result

- Methods can produce a result
 - To store, to use later, to pass to another part of our program, ...
 - E.g., method that calculates square of a number
- Need to:
 - Use specific *return type* like `int` or `String`
 - Include `return` statement

```
int getSquare() {  
    return 5 * 5  
}
```


Naming convention

Lower camel case

- First letter lowercase, additional words start with capital letter
- Examples:
 - `getSquare()`
 - `sayHello()`

Case sensitivity

Java is case sensitive!

If method is called `getSquare()` , `getsquare()` won't work!

Built-in vs. Custom methods

Built-in

- Called using the dot notation
 - `"JAVA".toLowerCase();`

Custom methods

- Called as-is
 - `sayHello();`

Using a custom method

```
class HelloThere {  
    public static void main(String[] args){  
        sayHello();  
    }  
    static void sayHello() {  
        System.out.println("Hello there!")  
    }  
}
```



Outside of the main method

"Hello there!"

Recap

```
class HelloMethod {  
    public static void main (String[] args) {  
        sayHello();  
        int fiveSquared = getSquare(); // Saving the result of getSquare() as int  
    }  
    static void sayHello() { // Just prints a message  
        System.out.println("Hello there!");  
    }  
    static int getSquare() { // Returns int  
        return 5*5;  
    }  
}
```

Hello there!

Let's practice!
INTERMEDIATE JAVA

Adding parameters

INTERMEDIATE JAVA



Jim White
Java Developer

Parameters

- Allow passing input values to methods
- Add flexibility
- Allow methods to respond to input
- Makes methods more reusable and useful

sayHello(String name)

```
static void sayHelloV1(){  
    System.out.println("Hello, there!");  
}
```

```
static void sayHello(String name){ // Add String parameter called name  
    System.out.println("Hello, " + name + "!"); // Use name in code  
}
```

Now, we can call `sayHello()` with different names:

```
sayHello("Alice"); // Prints "Hello, Alice!"  
sayHello("Sam"); // Prints "Hello, Sam!"
```


getSquare(int number)

```
static int getSquare(int number) {  
    return number * number  
}
```

```
getSquare(10); // Returns 100  
getSquare(2); // Returns 4
```

Calling methods with parameters

```
class HelloWorld {  
    public static void main (String[] args) {  
        sayHello("Alice");  
        int threeSquared = getSquared(3); // threeSquared = 9  
    }  
  
    static void sayHello(String name){  
        System.out.println("Hello, " + name + "!");  
    }  
  
    static int getSquare(int number) {  
        return number * number  
    }  
}
```

Hello, Alice!

More parameters - of any type

```
// Different types
static void printUser(String name, int age){
    System.out.println("User is " + name);
    System.out.println("They are " + age);
}
```

```
printUser("Ann", "32");
```

```
User is Ann
They are 32
```

```
// Same type, but more of them
static int addNumbers(int a, int b, int c){
    return a + b + c
}
```

```
int sum = addNumbers(2, 4, 6);
System.out.println(sum);
```

```
12
```

Recap

- Parameters allow us flexibility
- Parameters go in the parenthesis after method name
- We can have as many parameters as we want, also of different types

```
// Define method with parameters
static void printUser(String name, int age, String address){
    System.out.println("User is " + name);
    System.out.println("They are " + age);
    System.out.println("They live in " + address);
}
```

```
// Call the method
printUser("Sue", 43, "Canada");
```

Let's practice!
INTERMEDIATE JAVA